

Day 21–22: Strings

- String
- StringBuilder
- StringBuffer
- String methods
- Immutability

Practice:

- Reverse string
- Palindrome
- Anagram
- Character frequency

□ DAY 21 — STRING

1. First: String ante enti?

Simple ga cheppali ante:

String = Characters collection / sequence

Example:

"Hello"

Indulo:

H e l l o

5 characters unnayi.

Another example:

"Java"

"Hello World"

"12345"

"Welcome"

Ivanni Strings.

Real-life example

Oka person's name:

"Ravi"

Address:

"Hyderabad"

Message:

"Good Morning"

Ivi anni **String data**.

2. Java lo String ela create chestham?

Java lo String create cheyyadaniki common ga **2 ways** unnayi.

Method 1: String literal

String name = "Java";

Idi most commonly used method.

Method 2: new keyword

String name = new String("Java");

Rendu valid.

But difference undi. Adi **String Pool** concept ki related. Mundu basic ga understand chesukundham.

3. String is a class

Idi very important.

Java lo String primitive data type **kaadu**.

For example:

int

char

double

boolean

Ivi primitive data types.

But:

String

is a **class**.

Anduke:

String name = "Java";

ani rayagalamu.

String class Java lo already predefined ga untundi.

4. String declaration

String str;

Ikkada just variable declare chesam.

Value assign cheyyaledu.

String str = "Hello";

Ikkada:

- String → data type/class
 - str → variable
 - "Hello" → String value
-

5. String indexing

String lo characters ki indexes untayi.

Example:

String str = "HELLO";

Index:

Character: H E L L O

Index: 0 1 2 3 4

Important:

Java lo indexing **0 nunchi start avutundi**.

So:

H → 0

E → 1

L → 2

L → 3

O → 4

6. charAt()

Specific position lo unna character kavali ante:

```
String str = "Hello";
```

```
System.out.println(str.charAt(0));
```

Output:

H

Because index 0 lo H undi.

```
System.out.println(str.charAt(1));
```

Output:

e

charAt() gurinchi important point

charAt() **character ni return chestundi**, String ni kaadu.

```
char ch = str.charAt(0);
```

Correct.

7. String length

String lo enni characters unnayo telusukovali ante:

```
String str = "Hello";
```

```
System.out.println(str.length());
```

Output:

5

Important difference

Array:

`arr.length`

String:

`str.length()`

Array ki length property.

String ki `length()` method.

8. String + operator

Strings ni combine cheyyadaniki + use cheyyachu.

```
String first = "Hello";
```

```
String second = "World";
```

```
String result = first + second;
```

```
System.out.println(result);
```

Output:

HelloWorld

Space kavali ante:

```
String result = first + " " + second;
```

Output:

Hello World

9. String concatenation

Concatenation ante two or more Strings ni combine cheyyadam.

```
String a = "Java";
```

```
String b = "Programming";
```

```
String c = a + " " + b;
```

Result:

Java Programming

10. String and numbers

Interesting point.

```
int age = 20;
```

```
System.out.println("Age = " + age);
```

Output:

```
Age = 20
```

Java automatic ga number ni String representation ga combine chestundi.

Example:

```
System.out.println(10 + 20);
```

Output:

```
30
```

But:

```
System.out.println("10" + 20);
```

Output:

```
1020
```

Because "10" is a String.

❑ IMPORTANT: String methods

Day 21 lo most important topic idi.

11. length()

```
String s = "Java";
```

```
System.out.println(s.length());
```

Output:

12. charAt()

String s = "Java";

System.out.println(s.charAt(2));

Output:

v

Indexes:

J → 0

a → 1

v → 2

a → 3

13. toUpperCase()

String ni uppercase ki convert chestundi.

String s = "java";

System.out.println(s.toUpperCase());

Output:

JAVA

14. toLowerCase()

String s = "JAVA";

System.out.println(s.toLowerCase());

Output:

java

15. equals()

Two Strings same aa kaada check cheyyadaniki:

```
String a = "Java";
```

```
String b = "Java";
```

```
System.out.println(a.equals(b));
```

Output:

true

Different:

```
String a = "Java";
```

```
String b = "Python";
```

```
System.out.println(a.equals(b));
```

Output:

false

== vs equals()

Idi interviews lo very important.

==

Generally Strings ki == use cheste **references compare** chestundi.

equals()

String content compare chestundi.

Example:

```
String a = new String("Java");
```

```
String b = new String("Java");
```

```
System.out.println(a == b);
```

```
System.out.println(a.equals(b));
```

Output:

false

true

Why?

a and b separate objects.

But rendu objects lo content same:

Java

So:

a.equals(b)

→ true

Interview rule:

String content compare cheyyali ante equals() use cheyyali.

16. equalsIgnoreCase()

Uppercase/lowercase difference ignore chesi compare cheyyali ante:

String a = "JAVA";

String b = "java";

System.out.println(a.equalsIgnoreCase(b));

Output:

true

17. contains()

Oka String lo another String unda check chestundi.

String s = "Java Programming";

System.out.println(s.contains("Java"));

Output:

true

System.out.println(s.contains("Python"));

Output:

false

18. startsWith()

String particular text tho start avutunda?

String s = "Java Programming";

System.out.println(s.startsWith("Java"));

Output:

true

19. endsWith()

String s = "Java Programming";

System.out.println(s.endsWith("Programming"));

Output:

true

20. indexOf()

Oka character/String first occurrence index telusukovali ante:

String s = "Java";

System.out.println(s.indexOf('a'));

Output:

1

Because:

J a v a

0 1 2 3

21. lastIndexOf()

Last occurrence index:

```
String s = "Java";
```

```
System.out.println(s.lastIndexOf('a'));
```

Output:

3

22. substring()

String lo oka portion extract cheyyadaniki.

```
String s = "Programming";
```

```
System.out.println(s.substring(0, 4));
```

Output:

Prog

Important:

Start index included.

End index excluded.

substring(start, end)

0,4 ante:

0 1 2 3

Only these characters.

23. substring(start)

```
String s = "Programming";
```

```
System.out.println(s.substring(4));
```

Output:

ramming

Index 4 nunchi last varaku.

24. replace()

Characters or text replace cheyyadaniki:

```
String s = "Java";
```

```
System.out.println(s.replace('a', 'o'));
```

Output:

Jovo

Another example:

```
String s = "I like Java";
```

```
System.out.println(s.replace("Java", "Python"));
```

Output:

I like Python

25. trim()

Beginning and ending lo unnecessary spaces remove chestundi.

```
String s = " Java ";
```

```
System.out.println(s.trim());
```

Output:

Java

Note: Middle spaces ni remove cheyyadu.

26. isEmpty()

String empty aa kaada check chestundi.

```
String s = "";
```

```
System.out.println(s.isEmpty());
```

Output:

true

27. isBlank()

Java newer versions lo isBlank() whitespace-only String ni kuda check cheyyagaladu.

```
String s = "  ";
```

```
System.out.println(s.isBlank());
```

Output:

true

Difference:

isEmpty() → length 0 unda?

isBlank() → empty or only whitespace aa?

28. concat()

Strings combine cheyyadaniki:

```
String a = "Hello";
```

```
String b = "World";
```

```
System.out.println(a.concat(b));
```

Output:

HelloWorld

Space:

```
a.concat(" ").concat(b);
```

29. split()

String ni parts ga divide cheyyadaniki.

Example:

```
String s = "Java Python C++";
```

```
String[] arr = s.split(" ");
```

Result:

Java

Python

C++

Because space delimiter ga use chesam.

30. compareTo()

Two Strings ni lexicographically compare cheyyadaniki.

```
String a = "Apple";
```

```
String b = "Banana";
```

```
System.out.println(a.compareTo(b));
```

Output negative value.

Basic idea:

0 → both equal

negative → first String comes before second

positive → first String comes after second

Interview level lo idi useful.

□ DAY 21 — STRING IMMUTABILITY

Idi **very very important concept**.

Immutability ante enti?

Once String object create ayyaka, aa object content ni change cheyyalem.

Simple ga:

String object cannot be modified.

Example

```
String s = "Hello";
```

```
s.concat(" World");
```

```
System.out.println(s);
```

Output:

Hello

Enduku?

concat() original String ni modify cheyyadu.

Instead, new String create chestundi.

Correct:

```
s = s.concat(" World");
```

Now:

Hello World

□ Mind lo ila imagine chesko

Initially:

s

↓

"Hello"

concat() call chesinappudu:

"Hello" + " World"

New object:

"Hello World"

But s automatically new object ki point avvadu unless assign chesthe.

```
s = s.concat(" World");
```

Now:

s

↓

"Hello World"

Why String immutable?

Main reasons:

1. Security

Passwords, URLs, file paths etc. Strings widely use chestham.

2. String Pool

Same String literals ni reuse cheyyadaniki Java ki immutability useful.

3. Thread safety

Immutable object ni multiple threads share cheyyadam safer.

4. Hashing

Strings HashMap, HashSet etc. lo keys ga use chestham. Content change avvakapovadam important.

□ STRING POOL

Idi String concept lo important.

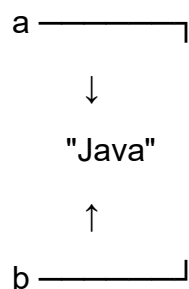
Suppose:

String a = "Java";

String b = "Java";

Java String literals kosam special memory area maintain chestundi, commonly **String Pool** ani antam.

Conceptually:



Same literal already pool lo unte Java reuse cheyyagaladu.

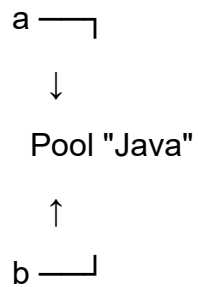
new String() difference

String a = "Java";

String b = "Java";

String c = new String("Java");

Conceptually:



c → separate String object

Anduke:

a == b

usually true for these same interned literals.

But:

a == c

false because c refers to a different object.

Still:

a.equals(c)

true.

Again:

Content compare → equals()

□ DAY 22 — STRINGBUILDER

Now problem enti?

String immutable.

Suppose manam loop lo repeatedly String modify chestunnam.

```
String s = "";
```

```
for (int i = 0; i < 1000; i++) {  
    s = s + i;  
}
```

Every concatenation potentially creates new String objects.

Large repeated modifications ki inefficient avvachu.

Appudu:

StringBuilder

use chestham.

31. StringBuilder ante enti?

StringBuilder is a mutable sequence of characters.

Mutable ante:

Existing object content ni modify cheyyagalamu.

Example:

```
StringBuilder sb = new StringBuilder("Hello");
```

```
sb.append(" World");
```

```
System.out.println(sb);
```

Output:

Hello World

Ikkada sb ni modify chesam.

32. append()

Most important method.

```
StringBuilder sb = new StringBuilder();
```

```
sb.append("Java");  
sb.append(" ");  
sb.append("Programming");
```

```
System.out.println(sb);
```

Output:

Java Programming

33. insert()

Middle lo insert cheyyadaniki:

```
StringBuilder sb = new StringBuilder("Jva");
```

```
sb.insert(1, "a");
```

```
System.out.println(sb);
```

Output:

Java

34. delete()

Characters delete cheyyadaniki:

```
StringBuilder sb = new StringBuilder("Java");
```

```
sb.delete(1, 3);
```

```
System.out.println(sb);
```

Output:

Ja

Because indexes 1 and 2 remove ayyayi.

35. deleteCharAt()

Single character remove:

```
StringBuilder sb = new StringBuilder("Java");
```

```
sb.deleteCharAt(1);
```

```
System.out.println(sb);
```

Output:

Jva

36. reverse()

StringBuilder lo direct ga reverse method undi.

```
StringBuilder sb = new StringBuilder("Java");
```

```
sb.reverse();
```

```
System.out.println(sb);
```

Output:

avaJ

37. setCharAt()

Particular index character change:

```
StringBuilder sb = new StringBuilder("Java");
```

```
sb.setCharAt(0, 'K');
```

```
System.out.println(sb);
```

Output:

Kava

Idi StringBuilder mutable ani prove chestundi.

38. **StringBuilder** → **String**

Sometimes StringBuilder result ni String ga kavali.

```
StringBuilder sb = new StringBuilder("Java");
```

```
String s = sb.toString();
```

Now:

sb → StringBuilder

s → String

□ **STRINGBUFFER**

StringBuffer kuda almost StringBuilder lagaane.

```
StringBuffer sb = new StringBuffer("Hello");
```

```
sb.append(" World");
```

```
System.out.println(sb);
```

Output:

Hello World

39. **StringBuilder** vs **StringBuffer**

Main difference:

Feature	StringBuilder	StringBuffer
Mutable	Yes	Yes
Faster generally	Yes	Usually
Thread-safe	No	Yes
Synchronization	No	Yes

Feature	StringBuilder	StringBuffer
Common use	Single-threaded code	Multi-threaded shared mutation

Easy memory trick:

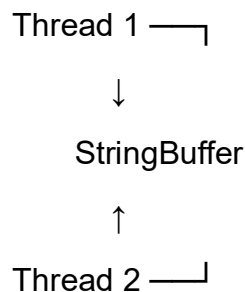
Builder = faster

Buffer = synchronized/thread-safe

Why StringBuffer slower?

StringBuffer methods are synchronized.

Example conceptually:



Synchronization helps control concurrent access, but it adds overhead.

StringBuilder synchronization provide cheyyadu.

□ String vs StringBuilder vs StringBuffer

This table **must remember**:

Feature	String	StringBuilder	StringBuffer
Mutable?	✗ No	✓ Yes	✓ Yes
Modification	New object	Same object modified	Same object modified
Speed for repeated modification	Lower	Higher	Lower than Builder generally
Thread-safe	Immutable, hence inherently safe to share	✗ No	✓ Yes
Common use	Normal text	Frequent changes	Thread-safe mutable text

Practice 1: Reverse String

Question:

Input: Java

Output: avaJ

Approach 1 — StringBuilder

Very easy:

```
String s = "Java";
```

```
StringBuilder sb = new StringBuilder(s);
```

```
System.out.println(sb.reverse());
```

Output:

avaJ

Approach 2 — Loop

Interview lo logic explain cheyyadaniki idi important.

```
String s = "Java";
```

```
String reverse = "";
```

```
for (int i = s.length() - 1; i >= 0; i--) {  
    reverse = reverse + s.charAt(i);  
}
```

```
System.out.println(reverse);
```

Let's understand.

String:

J a v a

0 1 2 3

length() = 4.

Last index:

$4 - 1 = 3$

So loop:

$i = 3 \rightarrow a$

$i = 2 \rightarrow v$

$i = 1 \rightarrow a$

$i = 0 \rightarrow J$

Result:

avaJ

Practice 2: Palindrome

Palindrome ante enti?

Forward and backward same unte palindrome.

Examples:

madam

level

radar

racecar

Non-palindrome:

hello

java

world

Logic

Suppose:

madam

Reverse:

madam

Both same.

Therefore palindrome.

Java program

```
String s = "madam";
```

```
String reverse = "";
```

```
for (int i = s.length() - 1; i >= 0; i--) {  
    reverse = reverse + s.charAt(i);  
}
```

```
if (s.equals(reverse)) {  
    System.out.println("Palindrome");  
} else {  
    System.out.println("Not Palindrome");  
}
```

Why equals()?

Because we want to compare String **content**.

```
s.equals(reverse)
```

Palindrome using StringBuilder

```
String s = "madam";
```

```
String reverse = new StringBuilder(s).reverse().toString();
```

```
if (s.equals(reverse)) {  
    System.out.println("Palindrome");  
} else {
```

```
System.out.println("Not Palindrome");  
}
```

Practice 3: Anagram

This is an important interview question.

Anagram ante enti?

Two Strings lo **same characters same frequency** lo unte, but order different ga undachu.

Examples:

listen

silent

Characters:

listen → l i s t e n

silent → s i l e n t

Same characters.

Therefore:

Anagram

Another:

triangle

integral

Not anagram

hello

world

Characters same kaavu.

Anagram approach 1: Sorting

Suppose:

listen

silent

Characters ni sort chesthe:

listen → eilnst

silent → eilnst

Both equal.

Therefore anagram.

Java logic

String a = "listen";

String b = "silent";

```
char[] arr1 = a.toCharArray();
```

```
char[] arr2 = b.toCharArray();
```

Now arrays:

arr1 → l i s t e n

arr2 → s i l e n t

Sort:

```
Arrays.sort(arr1);
```

```
Arrays.sort(arr2);
```

Now:

arr1 → e i l n s t

arr2 → e i l n s t

Compare:

```
Arrays.equals(arr1, arr2)
```

Full program:

```
import java.util.Arrays;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
String a = "listen";
String b = "silent";

char[] arr1 = a.toCharArray();
char[] arr2 = b.toCharArray();

Arrays.sort(arr1);
Arrays.sort(arr2);

if (Arrays.equals(arr1, arr2)) {
    System.out.println("Anagram");
} else {
    System.out.println("Not Anagram");
}
}
```

toCharArray() ante enti?

Very important.

```
String s = "Java";
```

```
toCharArray():
```

```
char[] arr = s.toCharArray();
```

Result:

```
arr[0] = 'J'
```

```
arr[1] = 'a'
```

```
arr[2] = 'v'
```

```
arr[3] = 'a'
```

So String ni character array ga convert chestundi.

Practice 4: Character Frequency

Idi interview lo very common.

Question:

Input: banana

Characters enni times unnayi?

b $\rightarrow$ 1

a $\rightarrow$ 3

n $\rightarrow$ 2

Simple approach: 26-size array

Lowercase English letters only assume chesthe:

String s = "banana";

```
int[] freq = new int[26];
```

```
for (int i = 0; i < s.length(); i++) {
```

```
    char ch = s.charAt(i);
```

```
    freq[ch - 'a']++;
```

```
}
```

□ ch - 'a' enti?

Idi first confusing ga untundi.

Suppose:

ch = 'a'

Then:

'a' - 'a' = 0

So a $\rightarrow$ index 0.

For:

'b' - 'a' = 1

So:

a → 0

b → 1

c → 2

...

z → 25

Therefore:

freq[ch - 'a']++;

means:

Aa character ki corresponding frequency index ni 1 increase cheyyi.

Example: banana

b → index 1

a → index 0

n → index 13

a → index 0

n → index 13

a → index 0

Final:

a → 3

b → 1

n → 2

Full character frequency program

String s = "banana";

int[] freq = new int[26];

```

for (int i = 0; i < s.length(); i++) {
    char ch = s.charAt(i);
    freq[ch - 'a']++;
}

for (int i = 0; i < 26; i++) {
    if (freq[i] > 0) {
        System.out.println((char)(i + 'a') + " : " + freq[i]);
    }
}

```

Output:

a : 3

b : 1

n : 2

□ Character frequency using HashMap

Real-world/general character frequency kosam HashMap very useful.

```
import java.util.HashMap;
```

```

public class Main {
    public static void main(String[] args) {

        String s = "banana";

        HashMap<Character, Integer> map = new HashMap<>();

        for (char ch : s.toCharArray()) {
            map.put(ch, map.getOrDefault(ch, 0) + 1);
        }
    }
}

```

```
        System.out.println(map);
    }
}
```

Concept:

$b \rightarrow 1$

$a \rightarrow 3$

$n \rightarrow 2$

getOrDefault() meaning

`map.getOrDefault(ch, 0)`

Ante:

ch already map lo unte current value ivvu; lekapothe 0 ivvu.

Then:

+ 1

frequency increase chestham.

 **Important: Spaces and uppercase**

Suppose:

"Hello World"

Question: spaces ni count cheyyala?

Adi problem statement meeda depend avutundi.

If spaces ignore cheyyali:

```
if (ch != ' ') {
```

```
    ...
```

```
}
```

Uppercase/lowercase same ga consider cheyyali ante:

```
s = s.toLowerCase();
```

Example:

"Java"

becomes:

"java"

Then frequency calculate cheyyachu.

□ MOST IMPORTANT STRING METHODS CHEAT SHEET

Method	Purpose
length()	Number of characters
charAt(i)	Character at index
equals()	Content comparison
equalsIgnoreCase()	Comparison ignoring case
toUpperCase()	Uppercase
toLowerCase()	Lowercase
contains()	Contains text?
startsWith()	Starts with text?
endsWith()	Ends with text?
indexOf()	First occurrence
lastIndexOf()	Last occurrence
substring()	Extract part
replace()	Replace characters/text
trim()	Remove leading/trailing spaces
isEmpty()	Length zero?
isBlank()	Empty/whitespace only?
concat()	Combine Strings
split()	Divide String
compareTo()	Lexicographical comparison
toCharArray()	String → char array

▣ STRINGBUILDER CHEAT SHEET

Method	Meaning
<code>append()</code>	Add at end
<code>insert()</code>	Insert at position
<code>delete()</code>	Delete range
<code>deleteCharAt()</code>	Delete one character
<code>reverse()</code>	Reverse
<code>setCharAt()</code>	Change character
<code>toString()</code>	Convert to String
<code>length()</code>	Current character count

Common mistakes students make

Mistake 1: `==` instead of `equals()`

Wrong for content comparison:

```
if (a == b)
```

Prefer:

```
if (a.equals(b))
```

Mistake 2: `length` instead of `length()`

Wrong:

```
s.length
```

Correct:

```
s.length()
```

Because String's length is a method.

Mistake 3: Forgetting index starts at 0

Java

is:

$J \rightarrow 0$

$a \rightarrow 1$

$v \rightarrow 2$

$a \rightarrow 3$

Mistake 4: Thinking String methods modify original String

Example:

```
String s = "hello";
```

```
s.toUpperCase();
```

```
System.out.println(s);
```

Output:

hello

Because String immutable.

Correct:

```
s = s.toUpperCase();
```

Now:

HELLO

Interview Questions You Must Know

Q1. Is String primitive or class?

Answer:

String is a **class**, not a primitive data type.

Q2. Is String mutable?

No.

String is **immutable**.

Q3. Why is String immutable?

Main reasons include:

- Security
- String Pool optimization
- Thread-safety benefits
- Stable hash codes for hashed collections

Q4. Difference between == and equals()?

== → reference identity comparison

equals() → content comparison

Q5. String vs StringBuilder?

String → immutable

StringBuilder → mutable

Frequent String modifications unte StringBuilder generally better.

Q6. StringBuilder vs StringBuffer?

StringBuilder → not synchronized, generally faster

StringBuffer → synchronized, thread-safe

Q7. What is palindrome?

Forward and reverse same.

Example:

madam

Q8. What is anagram?

Two Strings lo same characters same frequencies unte, order different unna parvaedu.

listen

silent

Day 21–22 Learning Order

Nuvvu practical ga prepare avvali ante **ee order lo practice cheyyi**:

Level 1 — Basics

1. String declaration
2. Indexing
3. length()
4. charAt()
5. +
6. concat()

Level 2 — Methods

7. equals()
8. equalsIgnoreCase()
9. toUpperCase()
10. toLowerCase()
11. contains()
12. startsWith()
13. endsWith()
14. indexOf()
15. lastIndexOf()
16. substring()
17. replace()
18. trim()
19. split()

Level 3 — Concepts

20. String immutability
21. String Pool

22. == vs equals()

Level 4 — Mutable Strings

23. StringBuilder

24. append()

25. insert()

26. delete()

27. reverse()

28. setCharAt()

Level 5

29. StringBuffer

30. StringBuilder vs StringBuffer

Level 6 — Coding Problems

31. Reverse String

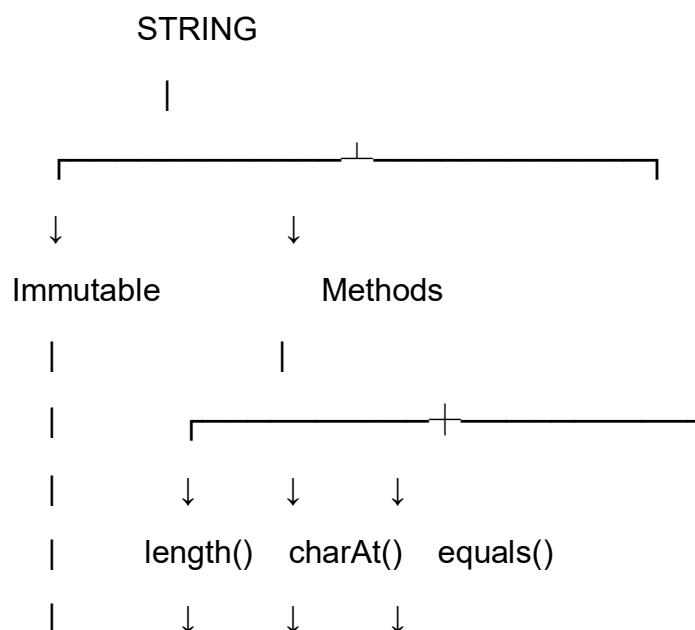
32. Palindrome

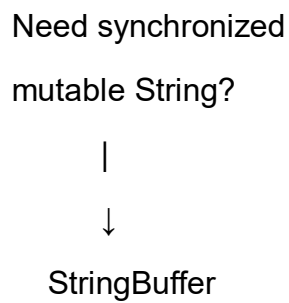
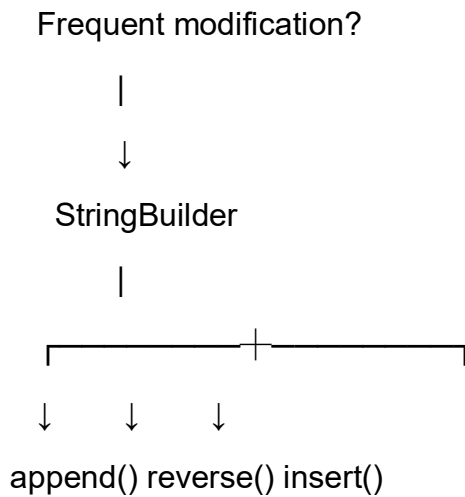
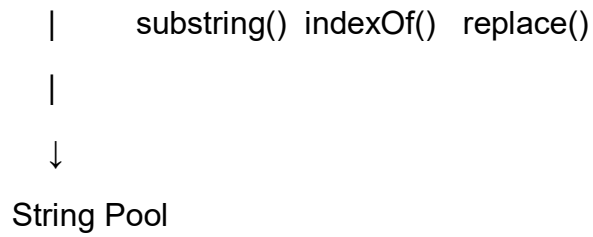
33. Anagram

34. Character frequency

□ One Big Picture

Finally ee entire topic ni **one diagram** laga mind lo petko:





☆ Final Memory Trick

Just remember these **6 lines**:

String = immutable text

StringBuilder = mutable + generally faster

StringBuffer = mutable + synchronized

equals() = content comparison

charAt() = one character

length() = number of characters

And coding problems:

Reverse → characters from last to first

Palindrome → String == reverse

Anagram → same characters + same frequencies

Frequency → count each character